

Stodoo - Documentation Administrateur

1. Vue d'Ensemble Technique

L'application **Stodoo** est une architecture "Serverless" hébergée sur **Google Cloud Platform (GCP)**. Elle agit comme un pont sécurisé entre les APIs de paiement (Stripe, PayPlug) et l'API XML-RPC d'Odoo.

Composants Clés :

- **Backend** : FastAPI (Python) tournant sur Google Cloud Run.
 - **Frontend** : React (Vite) servi par le backend.
 - **Base de Données** : Google Firestore (NoSQL).
 - **Sécurité** : Google Secret Manager pour la clé maîtresse de chiffrement.
-

2. Déploiement et Mise à Jour

Le déploiement est automatisé via un script PowerShell situé à la racine du projet : `deploy_prod.ps1`.

Étapes du script :

1. **Build Frontend** : Compile le code React dans le dossier `frontend/dist`.
2. **Synchronisation** : Copie le dossier `dist` vers `backend/app/static`.
3. **Deploy Cloud Run** : Envoie le code backend sur GCP, configure les variables d'environnement et les secrets.

Commande :

```
./deploy_prod.ps1
```

3. Gestion de la Sécurité et des Secrets

Toutes les données sensibles (clés API Stripe, Tokens PayPlug, mots de passe Odoo) sont **chiffrées** avant d'être stockées dans Firestore.

Clé Maîtresse (Master Key) :

- Le chiffrement utilise l'algorithme **AES-256 (Fernet)**.
- La clé est stockée dans Google Secret Manager sous le nom `stodoo-master-key`.
- Elle est injectée au démarrage du service via la variable d'environnement `MASTER_ENCRYPTION_KEY`.

⚠ **IMPORTANT** : Si cette clé est perdue ou modifiée dans Secret Manager, toutes les configurations existantes dans la base de données deviendront illisibles et devront être ressaisies.

4. Architecture des Données (Firestore)

Les données sont organisées en deux collections principales :

4.1. Collection clients

Chaque document représente un client. * `client_name` : String * `active` : Boolean * `auto_sync_active` : Boolean * `odoo_config` : Map (URL, DB, Username, encrypted_password, journal_mapping) * `stripe_config` : Map (encrypted_api_key) * `payplug_config` : Map (encrypted_token, auto_map_order, auto_map_partner) * `sync_state` : Map (last_sync_date) * `owner_uid` : ID de l'utilisateur Firebase propriétaire du dossier.

4.2. Collection users

Gère les accès à l'interface. * Le document doit avoir pour ID l'email de l'utilisateur. * Champ `role` : S'il est défini à "admin", l'utilisateur peut voir et modifier TOUS les clients de la plateforme. Sinon, il ne voit que ceux dont il est le `owner_uid`.

5. Automatisation (Cron)

La synchronisation automatique est déclenchée par **Google Cloud Scheduler**.

1. **Job** : `sync-all-clients-daily`

2. **Cible** : POST `https://[URL-CLOUD-RUN]/api/cron/sync_all`
 3. **Sécurité** : Le header `X-App-Engine-Cron: true` est vérifié par le backend pour s'assurer que l'appel provient bien de l'infrastructure Google.
 4. **Logique** : Le point de terminaison parcourt tous les clients ayant `active=True` ET `auto_sync_active=True`.
-

6. Monitoring et Logs

Pour surveiller l'état de santé du système : 1. Accédez à la console **Google Cloud > Logs Explorer**. 2. Filtrez par "Cloud Run Revision" et sélectionnez le service connecteurs-psp-odoo. 3. Recherchez les mots-clés "Cron", "Error" ou les noms des clients pour suivre le détail des exécutions.

7. Dépannage (Troubleshooting)

Erreur de déchiffrement

Si les logs affichent `InvalidToken` lors d'une tentative de synchro, cela signifie que la `MASTER_ENCRYPTION_KEY` ne correspond pas à celle utilisée lors de l'enregistrement des données. Vérifiez la configuration dans Secret Manager.

Erreur XML-RPC Odoo

Si la connexion échoue, vérifiez : 1. L'accessibilité de l'URL Odoo depuis l'extérieur. 2. Que l'utilisateur Odoo n'a pas activé la double authentification (2FA), ce qui bloque l'accès API standard (utiliser une "App Password" si disponible). 3. Que le journal Odoo possède bien le code devise attendu (EUR).